

The Mathematics of Cryptography Made Simple

Liz Burnell

1 Introduction

The RSA algorithm is one of the most popular encryption methodologies for ensuring the security of important information sent over the Internet. For example, if you go to <https://stuy.enschool.org> and open developers tools, navigate to security, security certificate, and then details, clicking on "Subject Public Key Algorithm" will show "PKCS 1 RSA Encryption," where PKCS means Public Key Cryptography Standards, published by RSA Laboratories.

RSA's popularity is rooted in its incredible security. Millions of people on the Internet know the algorithm's exact mechanics, and yet, it still works. How is this even possible? The short answer is that large numbers are hard to factor. The long answer will require us to delve into the modular arithmetic and the number theory behind it. Let's get into it!

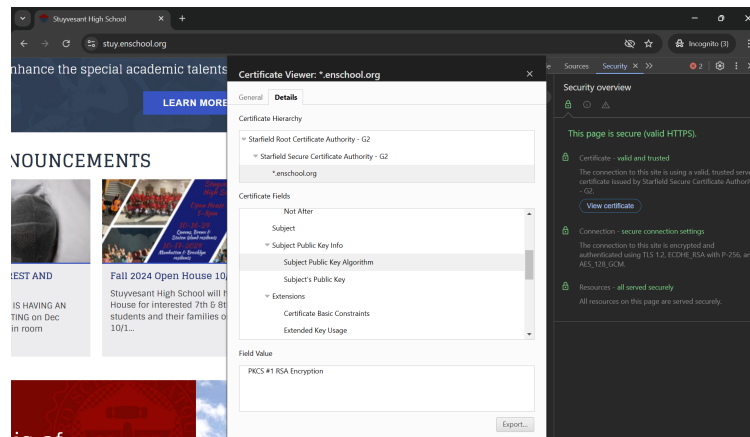


Figure 1: RSA Encryption is Used on Stuy's Website!

2 Back to Euler and Modular Math

1. **Modular math:** Modular arithmetic separates integers into classes based on their remainders when divided by a divisor, or modulus, n , which is denoted as follows:

$$a \equiv r \pmod{n}$$

For example, $7 \equiv 2 \pmod{5}$, meaning that 7 is congruent to (or in the same class as) 2 when 5 is used as the modulus.

2. **Euler's Totient Function:** Euler's Totient function plays a key role in RSA encryption and is denoted as $\phi(n)$. It counts the number of positive integers that are less than and relatively prime with n (these positive integers will include the number 1). For example $\phi(12) = 4$ because the integers 1, 5, 7, and 11 are co-prime with 12.

While for small numbers n , the corresponding $\phi(n)$ is somewhat easy to compute, this process becomes much harder, even for natural numbers only three digits large. This is because a prime factorization is needed to sort out all the numbers whose gcd with n is at least one of these primes.

3. **Euler's Theorem** is also key for RSA encryption. It states that for any two mutually prime positive integers a and n ,

$$a^{\phi(n)} \equiv 1 \pmod{n}.$$

The proof of the theorem would be understandable for a high-school student, but is omitted here as the focus of this article is slightly different.

3 A DIY Example

1. **Computational Steps Behind DIY RSA Encryption:**

Let's say you want to communicate with your distant cousin who is the sole inhabitant of Gao Island in Southeast Asia (this is far enough away that you wouldn't be able to send them a private message). You would need to make the message secure enough that if a third party got hold of it, they would not be able to decipher the message.

Fortunately, before they left, they agreed that they would communicate with you using RSA encryption. You know that all RSA encryption is based on choosing two prime numbers, so you decide to pick 97 and 103. We multiply them to get $n = 9991$. Since $\phi(9991) = (97 - 1) * (103 - 1) = 96 * 102 = 3 * 2^3 * 17 * 3 * 2$, you decide to set the encoding value e to the coprime number 5. You send a letter to your cousin telling him to use $n = 9991$ and $e = 5$ to encrypt his message. Knowing this, he encodes a secret message x by raising x to the power of 5 and calculating the remainder modulo 9991.

$$7048 \equiv x^5 \pmod{9991}.$$

On the message's voyage to you, the messenger turns out to be nosy and decides he wants to take a peak at your conversation. He sees you told your cousin to use $n = 9991$ and $e = 5$ to encrypt your message via RSA. He knows that to decrypt the message, he only needs to figure out what the factors of 9991 are. The messenger pulls out his advanced calculator and begins testing numbers. However, after ten minute of attempts, he has not been able to brute force an answer and decides it is a waste of time and gives up. When the message arrives to you, you are already aware that the factors of 9991 are 97 and 103, so you know that $\phi(n) = 96 * 102 = 9792$ and use your advanced calculator to determine the decoding value d by finding the multiplicative inverse of $e \pmod{\phi(n)}$: $d = e^{-1} \pmod{9792} = 3917$. Upon seeing the message 7048, you proceed to decode it by raising 7048 to the power of $d = 3917$ and computing the remainder when the modulus is 9991.

$$809 \equiv 7048^{3917} \pmod{9991}.$$

You realize your cousin's message was 0809, corresponding to the letters "h", "i" in the alphabet.

Step	Action
1	Select two big prime numbers p and q .
2	Multiply them to get n .
3	Calculate $\phi(n) = (p-1)(q-1)$.
4	Find integer e less than and mutually prime with $\phi(n)$. This is typically not easy.
5	Find e 's multiplicative inverse d such that $d \cdot e \equiv 1 \pmod{\phi(n)}$.

Table 1: RSA Encryption Steps

2. Why Does the Algorithm Work, From a Practical Perspective?

Both you and the messenger had advanced calculators. However, only you were able to decrypt your cousin's message. Why? All you had to do was compute the multiplicative inverse of e and do some straightforward calculations, which, while difficult for a person, would be easy for an advanced calculator to find. However, when the messenger got a hold of your messages, he had to figure out what the factors of 9991 were first in order to have the necessary information to determine the decryption key. However, there is no fast algorithm for finding these factors, and the only choice the messenger (and anyone else would have) had was to brute force test the factors. In the story above, the messenger gave up because this brute force approach was taking too much time. Analogously, in real life, two very large prime numbers are chosen, often hundreds of digits long, so that if anyone did intercept the message, it would take years for them to find the relevant prime factors, even with the best computing resources.

3. Why Does the Algorithm Work From a Mathematical Perspective?

We now understand why RSA is effective in practice. However, as readers of Stuyvesant's Math Survey Magazine, we are not just users of the algorithm. We are math enthusiasts who care about understanding the underlying mathematics that allow this algorithm to even work! So, what are the mathematics behind the encryption? Remember Euler's Theorem mentioned earlier? While the inventors of the RSA algorithm used Fermat's Little Theorem for their proof of the encryption technique, we can use Euler's theorem as well. We want to show that $m^{ed} \equiv m \pmod{n}$ where n is the product of two primes p, q and e and d are positive integers satisfying $ed \equiv 1 \pmod{\phi(n)}$. This is equivalent to setting $ed = 1 + k * \phi(n)$ where k is a positive integer (since e and d are positive primes, so their product must be greater than one). Assuming m is relatively prime to n , we have $m^{ed} = m^{1+k*\phi(n)} = m(m^{\phi(n)})^k$, and $m(m^{\phi(n)})^k \pmod{n} \equiv m(1)^k \pmod{n}$ by Euler's theorem, so $m^{ed} = m(m^{\phi(n)})^k \equiv m(1)^k \pmod{n} \equiv m \pmod{n}$.

4 Conclusion

The RSA algorithm is one of the most important inventions in Cryptography, forming the backbone of most secure communications over the Internet. Because RSA can both encrypt data and verify digital signatures, it is a fundamental component of Public Key Infrastructure (PKI), which creates, manages, and distributes digital certificates, which is essential for securing Internet communications. RSA has widespread use, protecting everything from email messages to online banking transactions. We hope that by reading this article, you were able to gain a better understanding of how the algorithm works, as well as a further appreciation for how this important component plays an important role in ensuring the security of the internet in our modern day!